

Reviewer Pack — Minimal Rank of Integral Quotient Sheaves Bounds Communicati...

Entry a6d65cd47ef8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 08:37:04 UTC

Minimal Rank of Integral Quotient Sheaves Bounds Communication Complexity for k-CNF

Entry ID: a6d65cd47ef8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 08:37:04 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Integral Quotient Sheaves) **Field B** (complexity object): Communication Complexity (k-CNF Satisfiability)

Statement:

[‘For every k-CNF formula with n variables, the communication complexity for its satisfiability problem is upper-bounded by $O(2^{r(n)})$, where $r(n)$ is the minimal rank of an integral quotient sheaf associated with the formula.’, ‘Equivalently, for a given 3-SAT instance I , there exists a constant c such that the number of communication steps required to solve I is at most c times the minimal rank $r(I)$ of the integral quotient sheaf.’, ‘Finally, if there exists an instance of k-CNF with communication complexity $O(2^{r(n)})$ and minimal rank $r(I) = n - 1$, then the conjecture is falsified.’]

Rationale (proposer’s reasoning):

[‘Integral quotient sheaves provide a geometric interpretation for counting certain types of configurations in algebraic varieties. If such sheaves could be used to bound communication complexity, it would suggest a deeper connection between algebraic geometry and complexity theory.’, ‘The rank of an integral quotient sheaf reflects the complexity of the underlying variety, which could potentially relate to the difficulty of solving the associated k-CNF instance.’, ‘This conjecture is motivated by the recent development in using geometric invariants for analyzing communication complexity problems.’]

Taxonomy category: geometric_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6bb8aaf4b77a5e3a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if all 30 seeds yield communication complexity $\leq c * r(n)$ for a constant c and minimal rank $r(n) \leq n - 1$, and falsified if any seed produces communication complexity $> c * r(n)$ or $r(n) > n - 1$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:(Integral Quotient Sheaves) AND communication complexity - title:(k-CNF Satisfiability) AND integral quotient sheaf-minimal rank (integral quotient sheaves) AND bounds communication complexity

Top relevant hits considered: - [http://arxiv.org/abs/hep-th/9809072v1] Complex Monopoles in the Path Integral - [http://arxiv.org/abs/1510.08876v2] Feynman integral in $\mathbb{R}^1 \oplus \mathbb{R}^m$ and complex expansion of ${}_2F_1$ - [http://arxiv.org/abs/2405.05873v4] Duality for Cohen-Macaulay Complexes through Combinatorial Sheaves

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 20 # Number of variables
    alpha = 0.5 # Clause density

    def generate_3sat_instance(n, alpha):
        clauses = []
        for _ in range(int(alpha * n * (n - 1) / 2)):
            clause = random.sample(range(1, n + 1), 3)
            if random.choice([True, False]):
                clause = [-x for x in clause]
            clauses.append(clause)
        return clauses

    def compute_minimal_rank(clauses):
        # Placeholder function to simulate computation of minimal rank
        # This is a dummy implementation and should be replaced with actual logic
        return len(clauses) # Simplified for testing purposes

    def communication_complexity(rank):
        # Placeholder function to simulate communication complexity
```

```

    # This is a dummy implementation and should be replaced with actual logic
    return 2 ** rank

clauses = generate_3sat_instance(n, alpha)
rank = compute_minimal_rank(clauses)
comm_complexity = communication_complexity(rank)

metric_name = "communication_complexity"
metric_value = comm_complexity
instances_tested = 1
conjecture_holds = False
counterexample = ""

if comm_complexity <= 2 * rank:
    conjecture_holds = True
else:
    counterexample = f"Communication complexity {comm_complexity} exceeds c*rank={2 * rank}"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ted': 1, 'conjecture_holds': False, 'counterexample': 'Communication complexity 39614081257132168796
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 39614081257132168796771975168, '
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 39614081257132168796771975168, '

```

TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 39614081257132168796771975168, '}'
 TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 39614081257132168796771975168, '}'
 TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 39614081257132168796771975168, '}'
 TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 39614081257132168796771975168, '}'
 TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 39614081257132168796771975168, '}'
 TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 39614081257132168796771975168, '}'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test has produced a counterexample where the communication complexity exceeds the constant c times the minimal rank $r(n)$. Specifically, the commun | next: Investigate the properties of integral quotient sheaves and k -CNF formulas that lead to such high communication complexities. Consider whether there are specific types of k -CNF formulas or conditions under which the conjecture does not hold.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12447
2	preregistration	ollama_remote	glm4:latest	0	0	5733
3	novelty	ollama_remote	glm4:latest	0	0	4681
4	novelty	ollama_remote	glm4:latest	0	0	5969
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42530
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7633
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8300
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8383
9	judge	ollama_remote	glm4:latest	0	0	11615

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 107291 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/a6d65cd47ef8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a6d65cd47ef8.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a6d65cd47ef8.tar.gz` (if generated)